

آزمایشگاه طراحی سیستم های دیجیتال

دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف



دانشکده مهندسی کامپیوتر دانشگاه صنعتی شریف

بهار ۱۴۰۵

استاد: دکتر انصاری

اعضای گروه:

کیمیا قدیر ۴۰۲۱۰۶۳۲۶

کیمیا رضائی ۴۰۲۱۰۶۰۴۸

ملینا طاهری ۴۰۲۱۰۶۱۶۷

گزارشکار آزمایش چهارم

موضوع آزمایش

طراحی پشته (Stack) با استفاده از توصیف رفتاری در Verilog

هدف آزمایش

هدف این آزمایش آشنایی با طراحی مدارات ترتیبی در Verilog و پیاده‌سازی ساختار پشته می‌باشد. همچنین در این آزمایش نحوه استفاده از سیگنال‌های کنترلی Push و Pop، مدیریت حافظه، استفاده از اشاره‌گر پشته و بررسی وضعیت‌های Full و Empty مورد مطالعه قرار می‌گیرد.

شرح کلی آزمایش

در این آزمایش یک پشته با ۸ خانه حافظه طراحی می‌شود که هر خانه قادر به ذخیره ۴ بیت داده است. پشته به صورت LIFO عمل می‌کند؛ یعنی آخرین داده‌ای که وارد پشته می‌شود، اولین داده‌ای است که خارج خواهد شد.

ورودی‌ها و خروجی‌های مدار به صورت زیر هستند:

Inputs:	Clk	Clock signal
	RstN	Reset signal
	Data_In	4-bit data into the stack
	Push	Push Command
	Pop	Pop Command
Outputs:	Data_Out	4-bit output data from stack
	Full	Full=1 indicates that the stack is full
	Empty	Empty=0 indicates that the stack is empty

برای طراحی این مدار از یک آرایه حافظه ۸ تایی استفاده شده است و توسط یک اشاره‌گر، محل ذخیره داده‌ها کنترل می‌شود. همچنین برای بررسی صحت عملکرد مدار، تست‌بنچ نوشته شده و حالات مختلف مانند Push، Pop، Full، Empty و Reset تست شده‌اند.

مراحل طراحی و پیاده‌سازی

۱- تعریف ماژول اصلی Stack

ابتدا ماژول اصلی طراحی شده و ورودی‌ها و خروجی‌های آن تعریف می‌شوند.

```
1  module stack (  
2      input Clk,  
3      input RstN,  
4      input Push,  
5      input Pop,  
6      input [3:0] Data_In,  
7      output reg [3:0] Data_Out,  
8      output reg Full,  
9      output reg Empty  
10 );
```

در این قسمت:

- ورودی‌های کنترلی و داده تعریف شده‌اند.

- خروجی‌ها به صورت reg تعریف شده‌اند زیرا در بلاک always مقداردهی می‌شوند.

۲- تعریف حافظه و اشاره‌گر پشته

برای ذخیره داده‌ها از یک حافظه ۸ خانه‌ای استفاده شده است.

```
12  reg [3:0] stack_pointer;  
13  reg [3:0] mem [7:0];  
14
```

در این بخش:

- آرایه mem شامل ۸ خانه حافظه ۴ بیتی است.

- متغیر stack_pointer محل فعلی بالای پشته را مشخص می‌کند.

۳- بلاک اصلی always

عملکرد اصلی مدار در بلاک always نوشته شده است.

```
always @(posedge Clk or negedge RstN)
```

این بلاک:

- در لبه مثبت کلاک اجرا می‌شود.

- در لبه منفی ریست نیز فعال می‌شود تا مدار ریست گردد.

۴- بخش Reset

در صورت فعال شدن ریست، تمام وضعیت‌های پشته به حالت اولیه بازمی‌گردند.

```
16     if (!RstN) begin
17         stack_pointer <= 0;
18         Empty <= 1;
19         Full <= 0;
20         Data_Out <= 0;
21     end
```

در این قسمت:

- اشاره‌گر صفر می‌شود.

- پشته خالی اعلام می‌شود.

- سیگنال Full صفر می‌شود.

- خروجی داده نیز صفر می‌شود.

۵- عملیات Push

در صورتی که Push فعال باشد و پشته پر نباشد، داده وارد پشته می‌شود.

```
2'b10: begin
    if (!Full) begin
        mem[stack_pointer] <= Data_In;
        stack_pointer <= stack_pointer + 1;
        Empty <= 0;
        if (stack_pointer == 4'd7)
            Full <= 1;
    end
end
```

در این قسمت:

- داده ورودی در محل فعلی حافظه ذخیره می‌شود.

- اشاره‌گر یک واحد افزایش پیدا می‌کند.

- سیگنال Empty صفر می‌شود.

- اگر آخرین خانه حافظه پر شود، سیگنال Full فعال می‌گردد.

- عملیات Pop

در صورت فعال بودن Pop و خالی نبودن پشته، آخرین داده خارج می‌شود.

```
2'b01: begin
    if (!Empty) begin
        Data_Out <= mem[stack_pointer - 1];
        stack_pointer <= stack_pointer - 1;
        Full <= 0;
        if (stack_pointer == 4'd1)
            Empty <= 1;
    end
end
```

در این بخش:

- آخرین داده ذخیره شده از پشته خوانده می شود.

- اشاره گر یک واحد کاهش پیدا می کند.

- سیگنال Full غیرفعال می شود.

- اگر آخرین داده خارج شود، سیگنال Empty فعال می شود.

۷- عملیات همزمان Push و Pop

اگر Push و Pop همزمان فعال شوند، شرایط مختلف بررسی می شود.

```
2'b11: begin
    if (!Empty && !Full) begin
        Data_Out <= mem[stack_pointer - 1];
        mem[stack_pointer - 1] <= Data_In;
    end
```

در این حالت:

- آخرین داده پشته خارج می شود.

- داده جدید جایگزین همان داده می شود.

- اندازه پشته تغییر نمی کند.

```
else if (Empty) begin
    mem[stack_pointer] <= Data_In;
    stack_pointer <= stack_pointer + 1;
    Empty <= 0;
end
else if (Full) begin
    Data_Out <= mem[stack_pointer - 1];
    stack_pointer <= stack_pointer - 1;
    Full <= 0;
end
```

اگر پشته خالی باشد، عملیات فقط مانند Push انجام می‌شود و اگر پشته پر باشد، عملیات مشابه Pop خواهد بود.

طراحی Testbench

برای بررسی عملکرد مدار، تست‌بنچ طراحی شده است.

تعریف Task ها

برای ساده‌تر شدن تست مدار، چند Task تعریف شده است.

Task مربوط به Push

```
18 task do_push(input [3:0] val);
19     begin
20         data_in = val; push = 1; pop = 0;
21         @(posedge clk); #1;
22     end
23 endtask
```

این Task:

- مقدار ورودی را روی Data_In قرار می‌دهد.

- سیگنال Push را فعال می‌کند.

Task مربوط به Pop

```
26 task do_pop;
27     begin
28         push = 0; pop = 1;
29         @(posedge clk); #1;
30     end
31 endtask
```

این Task:

- سیگنال Pop را فعال می‌کند.

- عملیات خروج داده را انجام می‌دهد.

Task مربوط به Idle

```
33     task idle;
34         begin
35             push = 0; pop = 0;
36             @(posedge clk); #1;
37         end
38     endtask
```

در این حالت:

- هیچ عملیاتی روی پشته انجام نمی‌شود.

بررسی تست‌های انجام شده

تست اول: ورود ۸ داده به پشته

```
$display("-- Push 8 items --");
do_push(1); do_push(2); do_push(3); do_push(4);
do_push(5); do_push(6); do_push(7); do_push(8);
```

هشت داده وارد پشته می‌شوند.

پس از ورود داده هشتم، سیگنال Full فعال می‌شود.

تست دوم: Push در حالت Full

```
$display("-- Push on Full (should be ignored) --");
do_push(9);
if (full != 1) $display("FAIL: Full should be 1");
```

چون پشته پر است، داده جدید وارد نمی‌شود.

مقدار Full برابر ۱ باقی می‌ماند.

تست سوم: خروج همه داده‌ها

```
$display("-- Pop all items --");  
repeat(8) do_pop;
```

تمام داده‌ها از پشته خارج می‌شوند.

در پایان سیگنال Empty فعال می‌شود.

تست چهارم: Pop در حالت Empty

```
$display("-- Pop on Empty (should be ignored) --");  
do_pop;  
if (empty != 1) $display("FAIL: Empty should be 1");
```

چون پشته خالی است، هیچ تغییری ایجاد نمی‌شود.

تست پنجم: Push و Pop همزمان

```
$display("-- Simultaneous Push & Pop (should be ignored) --");  
do_push(3); idle;  
push = 1; pop = 1; data_in = 5;  
@(posedge clk); #1;  
push = 0; pop = 0;
```

داده قبلی خارج شده و داده جدید جایگزین می‌شود.

تست ششم: Reset در حین کار

```
$display("-- Reset mid-operation --");  
do_push(4); do_push(5);  
reset = 0; #25; reset = 1; idle;  
if (empty != 1) $display("FAIL: should be empty after reset");
```

پشته در حین عملکرد ریست می‌شود.

تمام داده‌ها حذف شده و پشته خالی می‌گردد.

```

# -- Push 8 items --
# t=31 push=1 pop=0 din=1 | dout=0 Full=0 Empty=1
# t=50 push=1 pop=0 din=1 | dout=0 Full=0 Empty=0
# t=51 push=1 pop=0 din=2 | dout=0 Full=0 Empty=0
# t=71 push=1 pop=0 din=3 | dout=0 Full=0 Empty=0
# t=91 push=1 pop=0 din=4 | dout=0 Full=0 Empty=0
# t=111 push=1 pop=0 din=5 | dout=0 Full=0 Empty=0
# t=131 push=1 pop=0 din=6 | dout=0 Full=0 Empty=0
# t=151 push=1 pop=0 din=7 | dout=0 Full=0 Empty=0
# t=171 push=1 pop=0 din=8 | dout=0 Full=0 Empty=0
# t=190 push=1 pop=0 din=8 | dout=0 Full=1 Empty=0
# -- Push on Full (should be ignored) --
# t=191 push=1 pop=0 din=9 | dout=0 Full=1 Empty=0
# -- Pop all items --
# t=211 push=0 pop=1 din=9 | dout=0 Full=1 Empty=0
# t=230 push=0 pop=1 din=9 | dout=8 Full=0 Empty=0
# t=250 push=0 pop=1 din=9 | dout=7 Full=0 Empty=0
# t=270 push=0 pop=1 din=9 | dout=6 Full=0 Empty=0
# t=290 push=0 pop=1 din=9 | dout=5 Full=0 Empty=0
# t=310 push=0 pop=1 din=9 | dout=4 Full=0 Empty=0
# t=330 push=0 pop=1 din=9 | dout=3 Full=0 Empty=0
# t=350 push=0 pop=1 din=9 | dout=2 Full=0 Empty=0
# t=370 push=0 pop=1 din=9 | dout=1 Full=0 Empty=1
# -- Pop on Empty (should be ignored) --
# -- Simultaneous Push & Pop (should be ignored) --
# t=391 push=1 pop=0 din=3 | dout=1 Full=0 Empty=1
# t=410 push=1 pop=0 din=3 | dout=1 Full=0 Empty=0
# t=411 push=0 pop=0 din=3 | dout=1 Full=0 Empty=0
# t=431 push=1 pop=1 din=5 | dout=1 Full=0 Empty=0
# t=450 push=1 pop=1 din=5 | dout=3 Full=0 Empty=0
# -- Reset mid-operation --
# t=451 push=1 pop=0 din=4 | dout=3 Full=0 Empty=0
# t=471 push=1 pop=0 din=5 | dout=3 Full=0 Empty=0
# t=491 push=1 pop=0 din=5 | dout=0 Full=0 Empty=1
# t=516 push=0 pop=0 din=5 | dout=0 Full=0 Empty=1
# ** Note: $stop      : D:/term/DSDL/4/tb.v(80)
#   Time: 531 ps  Iteration: 0  Instance: /tb

```